

Monet GAN Project

This project tackles the unpaired image-to-image translation task of transforming ordinary landscape photos into Claude Monet-style paintings using a CycleGAN backbone. The base model employs dual generators and discriminators with cycle-consistency constraints so that a photo translated to Monet and back recovers its content, while adversarial training encourages the stylized output to mimic Monet's color palette and brushwork.

The workflow begins with exploratory data analysis of the Monet and photo datasets, then builds TensorFlow input pipelines and trains the base CycleGAN while tracking generator and discriminator losses, per-epoch image snapshots, and best checkpoints. Next, we run targeted hyperparameter sweeps to select stable learning rates and optimizer settings, evaluate model quality with FID/MiFID metrics, and finally compare the base system against Least Squares, Hinge, and Wasserstein GAN variants to understand how different adversarial objectives impact style realism and content preservation.

Data Summary

Data Source

- Data was imported from the 'I'm Something of a Painter Myself' Kaggle competition.
- 2 folders provided from Kaggle: monet.jpg and photo.jpg. These will be converted to Tensorflow record files so it's easier to do work with

Reference:

Amy Jang, Ana Sofia Uzsoy, and Phil Culliton. I'm Something of a Painter Myself. <https://kaggle.com/competitions/gan-getting-started>, 2020. Kaggle.

Import Python Libraries

```
In [4]: import os
import glob
import tempfile
import zipfile
import shutil
import gc
import random
from IPython.display import Image
import cv2
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.applications.inception_v3 import InceptionV3, preprocess_input
```

Configure Constant Variables

- Define the exogenous variables that will be used throughout the modeling and analysis.

```
In [8]: # Set image/channel dimensions and training constants
IMG_SIZE = 256
INCEPTION_IMG_SIZE = 299
CHANNELS = 3
BATCH_SIZE = 1
MAX_EPOCHS = 20 # Max number of epochs to train for each GAN model
MAX_PHOTOS = 3500 # Select a subset of photos to reduce training time

AUTOTUNE = tf.data.AUTOTUNE # Tensorflow autotune

# Define where to save models
CHECKPOINT_DIR = './checkpoints'
BEST_MONET_GENERATOR_PATH = os.path.join(CHECKPOINT_DIR, 'monet_generator_best.weights.h5')

random.seed(666)
```

Import Datasets

- Data is imported from the saved photo and Monet files as provided by the Kaggle competition website

```
In [9]: # Data Loading
photo_ds_path = './photo.jpg'
monet_ds_path = './monet.jpg'

# Define file paths
monet_images_paths = sorted(glob.glob(os.path.join(monet_ds_path, '*.jpg')))
photo_images_paths = sorted(glob.glob(os.path.join(photo_ds_path, '*.jpg')))
print(f'Count of Monet paintings = {len(monet_images_paths):,}\nCount of photo images = {len(photo_images_paths):,}\n')

# Optionally cap the photo dataset size for faster experiments
if len(photo_images_paths) > MAX_PHOTOS:
    photo_images = sorted(random.sample(photo_images_paths, MAX_PHOTOS))
    print(f'Randomly sampled {len(photo_images):,} photos for training')
else:
    photo_images = photo_images_paths
```

Count of Monet paintings = 300

Count of photo images = 7,038

Randomly sampled 3,500 photos for training

EDA

RGB Histograms

- Visualize the average per-channel RGB histograms of the Monet's vs the photos

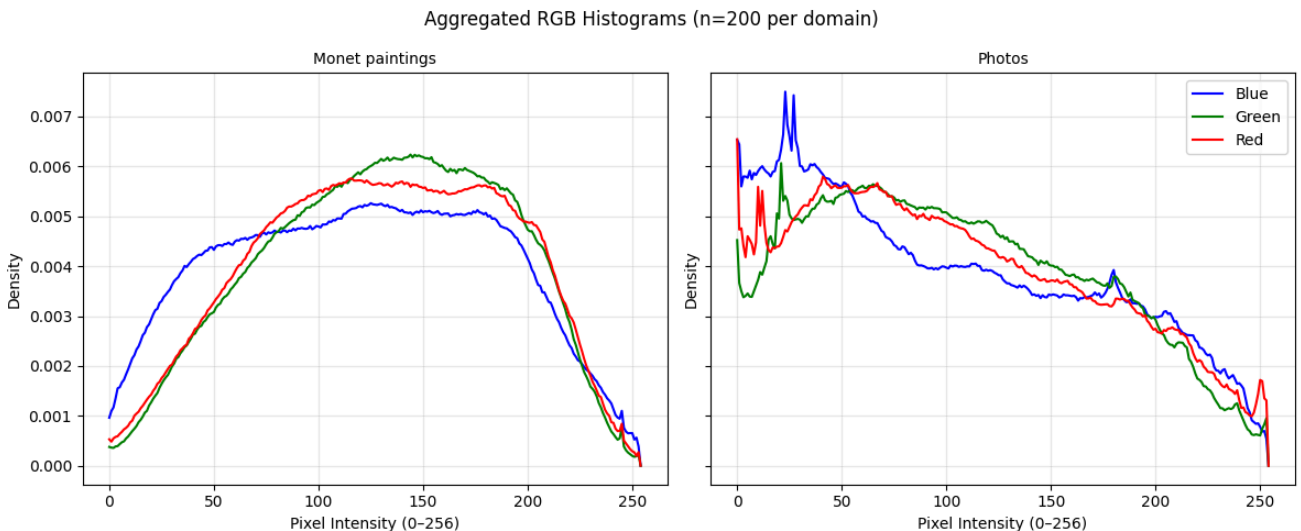
```
In [16]: # Cell 12: RGB histograms for Monet paintings vs photos
SAMPLE_SIZE = min(200, len(monet_images_paths), len(photo_images))
monet_files = random.sample(monet_images_paths, SAMPLE_SIZE)
photo_files = random.sample(photo_images, SAMPLE_SIZE)

def get_histogram_data(files):
    histograms = np.zeros((3, 256), dtype=np.float64)
    for path in files:
        img = cv2.imread(path, cv2.IMREAD_COLOR)
        for ch in range(3):
            histograms[ch] += cv2.calcHist([img], [ch], None, [256], [1, 256]).ravel()
    return histograms

def to_probability(histograms):
    return histograms / histograms.sum(axis=1, keepdims=True)

monet_prob = to_probability(get_histogram_data(monet_files))
photo_prob = to_probability(get_histogram_data(photo_files))

fig, axes = plt.subplots(1, 2, figsize=(12, 5), sharex=True, sharey=True)
for data, ax, title in [
    (monet_prob, axes[0], "Monet paintings"),
    (photo_prob, axes[1], "Photos"),
]:
    ax.plot(data[0], 'b', label='Blue')
    ax.plot(data[1], 'g', label='Green')
    ax.plot(data[2], 'r', label='Red')
    ax.set_title(title, fontdict={'fontsize': 10})
    ax.set_xlabel('Pixel Intensity (0-256)')
    ax.set_ylabel('Density')
    ax.grid(True, alpha=0.3)
axes[1].legend()
plt.suptitle(f'Aggregated RGB Histograms (n={SAMPLE_SIZE:,} per domain)')
plt.tight_layout()
plt.show()
```



Monet paintings show smoother, mid-tone-heavy distributions across RGB with gentle peaks around the 140–170 range and a shared roll-off at high intensities, implying a flatter, more balanced palette. **Photos** have sharper low-intensity spikes (especially in blue), broader spread, and a slower decline through the mid/high range, reflecting higher contrast and more varied lighting.

Similar Looking Pictures

- Find the best unique matches between 5 random Monet paintings and the photo set based on how correlated their RGB histograms are

```
In [22]: # Cell 15: Faceted RGB correlation grid (Monets with top photo matches)
SAMPLE_MONETS = min(8, len(monet_images_paths))
TOP_K_PHOTOS = 5
BINS = 256

# Load an image as RGB using cv2 (which reads BGR by default)
def _load_rgb(path):
    img_bgr = cv2.imread(path, cv2.IMREAD_COLOR)
    return cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
```

```

# Compute a normalized RGB histogram vector for correlation
# Concatenate per-channel probabilities into one feature vector
_def_eps = 1e-8

def _rgb_hist_vector(path, bins=BINS):
    img = cv2.imread(path, cv2.IMREAD_COLOR)
    hist = []
    for ch in range(3):
        hist = cv2.calcHist([img], [ch], None, [bins], [0, 256]).ravel()
        hist = hist / (hist.sum() + _def_eps)
        hist.append(hist)
    return np.concatenate(hist)

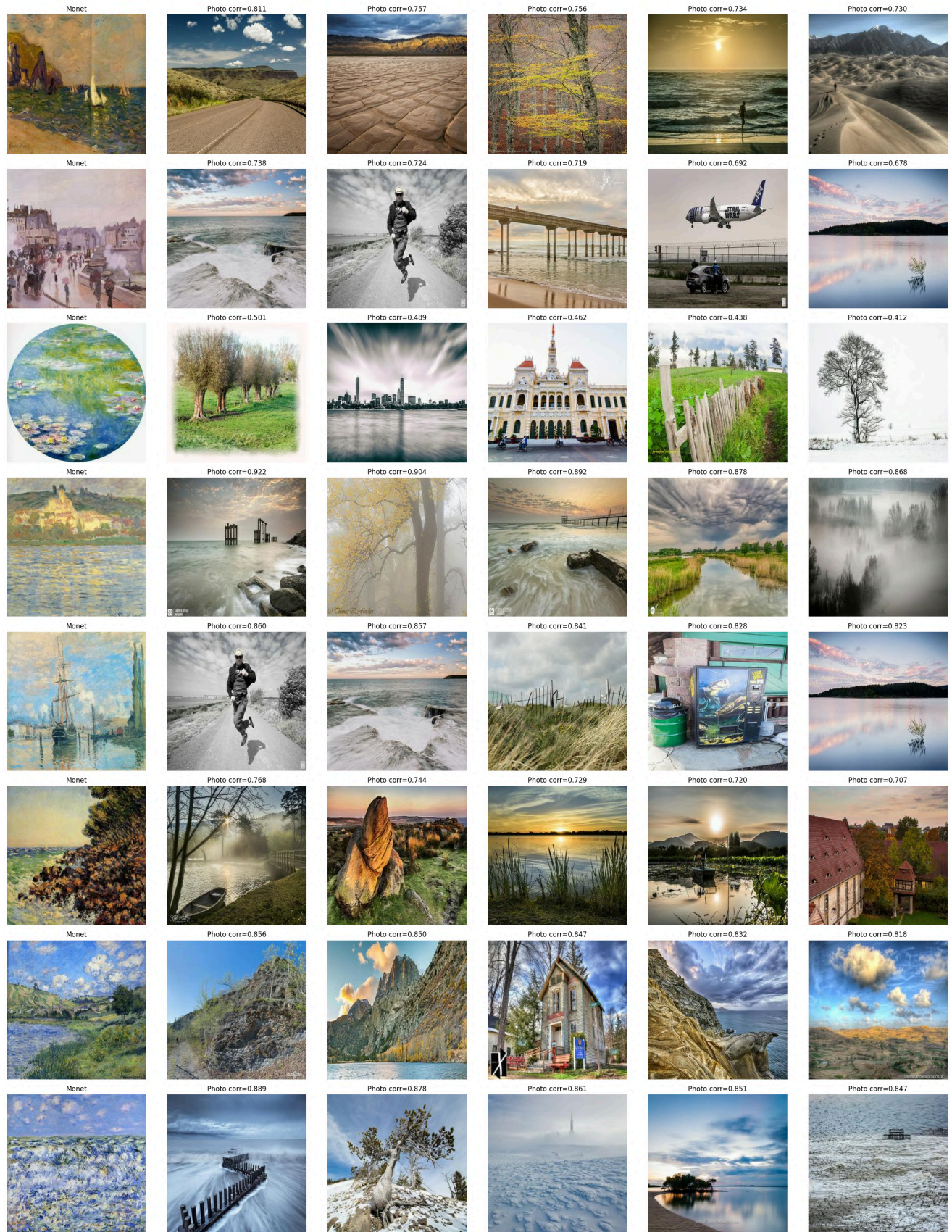
# Precompute photo histogram vectors to avoid reloading for every Monet
photo_hist_cache = [(p, _rgb_hist_vector(p)) for p in photo_images]

# Sample Monets and find top correlated photos by RGB histogram correlation
target_monets = random.sample(monet_images_paths, SAMPLE_MONETS)
correlation_rows = []
for monet_path in target_monets:
    monet_vec = _rgb_hist_vector(monet_path)
    scores = []
    for photo_path, photo_vec in photo_hist_cache:
        corr = np.corrcoef(monet_vec, photo_vec)[0, 1]
        scores.append((corr, photo_path))
    top_matches = sorted(scores, key=lambda x: x[0], reverse=True)[:TOP_K_PHOTOS]
    correlation_rows.append((monet_path, top_matches))

# Plot grid: Monet in first column, followed by top 5 correlated photos
fig, axes = plt.subplots(SAMPLE_MONETS, TOP_K_PHOTOS + 1, figsize=(4 * (TOP_K_PHOTOS + 1), 4 * SAMPLE_MONETS))
if SAMPLE_MONETS == 1:
    axes = [axes]
for row_idx, (monet_path, matches) in enumerate(correlation_rows):
    row_axes = axes[row_idx]
    monet_ax = row_axes[0]
    monet_ax.imshow(_load_rgb(monet_path))
    monet_ax.set_title('Monet')
    monet_ax.axis('off')
    for col_offset, (corr, photo_path) in enumerate(matches, start=1):
        ax = row_axes[col_offset]
        ax.imshow(_load_rgb(photo_path))
        ax.set_title(f'Photo corr={corr:.3f}')
        ax.axis('off')
plt.suptitle('Top photo matches by RGB histogram correlation', fontsize=14)
plt.tight_layout(rect=(0, 0, 1, 0.97))
plt.show()

```


Top photo matches by RGB histogram correlation



The plot shows that simple RGB histogram matching pulls photos with similar overall color palettes and tonal balance to each Monet, yielding modest but consistent correlations (roughly 0.7–0.9) across diverse scenes. Matches are driven by shared hues and luminance rather than fine structure, so semantically different photos can still align when their color distributions resemble the target painting.

Create Datasets

- Normalize/denormalize images and build a shuffled & batched tf.data pipeline that pairs each photo with a Monet painting so the GAN trains on aligned image pairs

```
In [5]: # Pick reference photos for qualitative comparisons
comparison_photo_paths = random.sample(photo_images, min(3, len(photo_images)))

# Track Monet outputs per epoch for visualization
epoch_snapshots = {}

# Track generator/discriminator losses across epochs
epoch_loss_history = {"generator": [], "discriminator": []}

# Prepare Monet paths as tensors for tf.data
monet_files_tensor = tf.constant(monet_images_paths)
monet_count = tf.cast(tf.shape(monet_files_tensor)[0], tf.int64)

# Load and normalize JPEG input
def load_image(path):
    img = tf.io.read_file(path)
    img = tf.image.decode_jpeg(img, channels=CHANNELS)
    img = tf.image.resize(img, [IMG_SIZE, IMG_SIZE])
    img = (tf.cast(img, tf.float32) / 127.5) - 1.0
    return img

# Convert tensor back to display range
def denormalize_image(img):
    img = (img + 1.0) / 2.0
    return tf.clip_by_value(img, 0.0, 1.0)

# Pair shuffled photos with Monet references
def map_pair(idx, photo_path):
    monet_idx = tf.math.floormod(idx, monet_count)
    monet_path = tf.gather(monet_files_tensor, monet_idx)
    photo_img = load_image(photo_path)
    monet_img = load_image(monet_path)
    return photo_img, monet_img

# Build shuffled training dataset
dataset = tf.data.Dataset.from_tensor_slices(photo_images)
dataset = dataset.shuffle(buffer_size=len(photo_images), reshuffle_each_iteration=True)
dataset = dataset.enumerate()
dataset = dataset.map(map_pair, num_parallel_calls=AUTOTUNE)
dataset = dataset.batch(BATCH_SIZE, drop_remainder=True)
dataset = dataset.prefetch(AUTOTUNE)
```

```
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1763522587.489134    13732 gpu_device.cc:2020] Created device /job:localhost/replica:0/task:0/device:GPU:0 with 1756 MB memory: -> device:
0, name: NVIDIA GeForce RTX 3050 Ti Laptop GPU, pci bus id: 0000:01:00:00, compute capability: 8.6
```

Describe Model Architecture

- A CycleGAN model was selected because is a natural fit for translating photos into Monet-style paintings. This is because it learns two reciprocal mappings `photo → Monet` and `Monet → photo` without needing paired training examples, which are impossible to obtain for this task. The cycle-consistency constraint enforces that translating a photo to Monet and back again recovers the original, preserving scene structure while the adversarial losses push outputs to match Monet's color palette and brushstroke distribution. Together, these properties let CycleGAN capture Monet's style faithfully while retaining the content of real-world photos.

Modeling

Helper Functions

- Helper functions that are used throughout the project to aid in modeling.

```
In [6]: # Produce Monet-style predictions for each photo path
def generate_monet_batch(photo_paths, generator=None):
    generator = generator or monet_generator
    outputs = []
    for path in photo_paths:
        photo = load_image(path)
        monet_pred = generator(tf.expand_dims(photo, axis=0), training=False)
        monet_img = tf.squeeze(monet_pred, axis=0)
        outputs.append(denormalize_image(monet_img).numpy())
    return outputs

# Visualize snapshots collected across epochs
def show_epoch_progression(photo_paths, snapshots):
    epochs = sorted(snapshots.keys())
    sample_count = len(photo_paths)
    fig, axes = plt.subplots(sample_count, len(epochs) + 1, figsize=(4 * (len(epochs) + 1), 4 * sample_count))
    if sample_count == 1:
        axes = [axes]
    original_images = [denormalize_image(load_image(path)).numpy() for path in photo_paths]
    for row_idx, photo_img in enumerate(original_images):
        row_axes = axes[row_idx]
        row_axes[0].imshow(photo_img)
        row_axes[0].set_title('Original Photo')
        row_axes[0].axis('off')
        for col_idx, epoch in enumerate(epochs, start=1):
            ax = row_axes[col_idx]
```

```

        ax.imshow(snapshots[epoch][row_idx])
        ax.set_title(f'Epoch {epoch}')
        ax.axis('off')
    fig.suptitle('Original vs. Monet-style progression', fontsize=14)
    plt.tight_layout(rect=(0, 0, 1, 0.97))
    plt.show()

# Plot generator and discriminator Loss history
def plot_loss_history(loss_history, model_name=None):
    gen_losses = loss_history["generator"]
    disc_losses = loss_history["discriminator"]
    epochs = range(1, len(gen_losses) + 1)
    plt.figure(figsize=(8, 4))
    plt.plot(epochs, gen_losses, label='Generator Loss')
    plt.plot(epochs, disc_losses, label='Discriminator Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    title = f"{model_name} - Generator vs Discriminator Loss" if model_name else 'Generator vs Discriminator Loss'
    plt.title(title)
    plt.legend()
    plt.grid(True, linestyle='--', alpha=0.3)
    plt.tight_layout()
    plt.show()

# Lazily Load InceptionV3 for FID computation
def _get_inception_model():
    _inception_model = InceptionV3(
        include_top=False,
        weights='imagenet',
        pooling='avg',
        input_shape=(INCEPTION_IMG_SIZE, INCEPTION_IMG_SIZE, 3)
    )
    return _inception_model

# Prepare images for Inception feature extraction
def _prepare_images_for_inception(images):
    images = tf.convert_to_tensor(images, dtype=tf.float32)
    images = tf.image.resize(images, [INCEPTION_IMG_SIZE, INCEPTION_IMG_SIZE])
    images = preprocess_input(images * 255.0)
    return images

# Compute Inception activations for a batch of images
def _get_inception_activations(images, batch_size=16):
    model = _get_inception_model()
    activations = []
    for idx in range(0, len(images), batch_size):
        batch = _prepare_images_for_inception(images[idx:idx + batch_size])
        activations.append(model(batch, training=False).numpy())
    return np.concatenate(activations, axis=0)

# Calculate mean and covariance of activations
def _calculate_activation_stats(activations):
    mu = np.mean(activations, axis=0)
    sigma = np.cov(activations, rowvar=False)
    return mu, sigma

# Compute Fréchet distance between distributions
def _calculate_frechet_distance(mu1, sigma1, mu2, sigma2, eps=1e-6):
    cov_prod = tf.matmul(
        tf.convert_to_tensor(sigma1 + np.eye(sigma1.shape[0]) * eps, dtype=tf.float64),
        tf.convert_to_tensor(sigma2 + np.eye(sigma2.shape[0]) * eps, dtype=tf.float64)
    )
    covmean = tf.linalg.sqrtm(cov_prod)
    if tf.math.reduce_any(tf.math.is_nan(covmean)):
        cov_prod += tf.eye(tf.shape(cov_prod)[0], dtype=tf.float64) * eps
    covmean = tf.linalg.sqrtm(cov_prod)
    covmean = tf.math.real(covmean)
    diff = mu1 - mu2
    trace = np.trace(sigma1 + sigma2 - 2.0 * covmean.numpy())
    fid = float(np.dot(diff, diff) + trace)
    return fid

# Estimate memorization term for MiFID
def _calculate_memorization_term(real_acts, gen_acts, batch_size=32):
    real_tensor = tf.convert_to_tensor(real_acts, dtype=tf.float32)
    gen_tensor = tf.convert_to_tensor(gen_acts, dtype=tf.float32)
    min_dists = []
    for idx in range(0, gen_tensor.shape[0], batch_size):
        g_batch = gen_tensor[idx:idx + batch_size]
        diff = tf.expand_dims(g_batch, axis=1) - tf.expand_dims(real_tensor, axis=0)
        dists = tf.reduce_sum(tf.square(diff), axis=-1)
        min_d = tf.sqrt(tf.reduce_min(dists, axis=1))
        min_dists.append(min_d)
    return float(tf.reduce_mean(tf.concat(min_dists, axis=0)).numpy())

# Compute FID and MiFID metrics for a generator
def calculate_fid_and_mifid(generator, num_samples=128, batch_size=8, lambda_mem=1.0):
    eval_photo_paths = random.sample(photo_images, min(num_samples, len(photo_images)))
    eval_monet_paths = random.sample(monet_images_paths, min(num_samples, len(monet_images_paths)))
    gen_images = []
    for path in eval_photo_paths:
        photo = load_image(path)
        monet_pred = generator(tf.expand_dims(photo, axis=0), training=False)
        monet_img = tf.squeeze(monet_pred, axis=0)

```

```

        gen_images.append(denormalize_image(monet_img).numpy())
    real_images = [denormalize_image(load_image(path)).numpy() for path in eval_monet_paths]
    gen_images = np.array(gen_images, dtype=np.float32)
    real_images = np.array(real_images, dtype=np.float32)
    real_acts = _get_inception_activations(real_images, batch_size=batch_size)
    gen_acts = _get_inception_activations(gen_images, batch_size=batch_size)
    mu_real, sigma_real = _calculate_activation_stats(real_acts)
    mu_gen, sigma_gen = _calculate_activation_stats(gen_acts)
    fid = _calculate_frechet_distance(mu_real, sigma_real, mu_gen, sigma_gen)
    memorization_term = _calculate_memorization_term(real_acts, gen_acts)
    mifid = fid + lambda_mem * memorization_term
    return {
        'fid': fid,
        'mifid': mifid,
        'memorization': memorization_term,
        'num_real': len(real_images),
        'num_generated': len(gen_images)
    }

# Measure combined generator and discriminator loss
def _combined_loss_metric(loss_G, loss_D):
    return float(loss_G + loss_D)

# Measure absolute difference between GAN losses
def _difference_loss_metric(loss_G, loss_D):
    return abs(float(loss_G - loss_D))

# Run a generic training loop with snapshot tracking
def run_generator_training_session(
    session_name,
    epochs,
    dataset_factory,
    steps_per_epoch,
    train_step_fn,
    snapshot_generator=None,
    snapshot_interval=1,
    snapshot_visualizer=None,
    best_selector=None,
):
    history = {'generator': [], 'discriminator': []}
    snapshots = {}
    best_epoch = 0
    best_metric = float('inf')
    best_payload = None

    for epoch in range(epochs):
        print(f"{session_name}: epoch {epoch + 1}/{epochs}")
        epoch_dataset = dataset_factory()
        progbar = tf.keras.utils.Progbar(target=steps_per_epoch, verbose=1)
        for step, (real_photo, real_monet) in enumerate(epoch_dataset.take(steps_per_epoch), start=1):
            loss_G, loss_D = train_step_fn(real_photo, real_monet)
            progbar.update(step, values=[('loss_G', float(loss_G)), ('loss_D', float(loss_D))])
            history['generator'].append(float(loss_G))
            history['discriminator'].append(float(loss_D))
        print(f"{session_name} epoch {epoch + 1} complete - loss_G: {float(loss_G):.3f}, loss_D: {float(loss_D):.3f}")

        if snapshot_generator and ((epoch + 1) % snapshot_interval == 0):
            epoch_snapshot = snapshot_generator()
            snapshots[epoch + 1] = epoch_snapshot
            if snapshot_visualizer:
                snapshot_visualizer(epoch + 1, epoch_snapshot)

        if best_selector:
            metric_value = best_selector['metric_fn'](loss_G, loss_D)
            if metric_value < best_metric:
                best_metric = metric_value
                best_epoch = epoch + 1
                if best_selector.get('on_best'):
                    best_payload = best_selector['on_best'](epoch + 1, loss_G, loss_D)
                if best_selector.get('message_fn'):
                    best_selector['message_fn'](epoch + 1, metric_value)

    return {
        'history': history,
        'snapshots': snapshots,
        'best_epoch': best_epoch,
        'best_metric': best_metric,
        'best_payload': best_payload,
    }

```

Generator and Discriminator

- Defining the generator and discriminator architecture

```

In [7]: # Define generator architecture
def build_generator():
    inputs = tf.keras.Input(shape=(IMG_SIZE, IMG_SIZE, CHANNELS))
    x = tf.keras.layers.Conv2D(64, 7, padding='same')(inputs)
    x = tf.keras.layers.LeakyReLU()(x)
    x = tf.keras.layers.Conv2D(128, 3, strides=2, padding='same')(x)
    x = tf.keras.layers.LeakyReLU()(x)
    x = tf.keras.layers.Conv2D(256, 3, strides=2, padding='same')(x)

```

```

x = tf.keras.layers.LeakyReLU()(x)
x = tf.keras.layers.Conv2DTranspose(128, 3, strides=2, padding='same')(x)
x = tf.keras.layers.LeakyReLU()(x)
x = tf.keras.layers.Conv2DTranspose(64, 3, strides=2, padding='same')(x)
x = tf.keras.layers.LeakyReLU()(x)
outputs = tf.keras.layers.Conv2D(3, 7, padding='same', activation='tanh')(x)
return tf.keras.Model(inputs, outputs, name='generator')

# Define discriminator architecture
def build_discriminator():
    inputs = tf.keras.Input(shape=(IMG_SIZE, IMG_SIZE, CHANNELS))
    x = tf.keras.layers.Conv2D(64, 4, strides=2, padding='same')(inputs)
    x = tf.keras.layers.LeakyReLU()(x)
    x = tf.keras.layers.Conv2D(128, 4, strides=2, padding='same')(x)
    x = tf.keras.layers.LeakyReLU()(x)
    x = tf.keras.layers.Conv2D(256, 4, strides=2, padding='same')(x)
    x = tf.keras.layers.LeakyReLU()(x)
    outputs = tf.keras.layers.Conv2D(1, 4, strides=1, padding='same')(x)
    return tf.keras.Model(inputs, outputs, name='discriminator')

# Instantiate Monet generator network
monet_generator = build_generator()
photo_generator = build_generator()
monet_discriminator = build_discriminator()
photo_discriminator = build_discriminator()

```

Train Base CycleGAN Model

- The train_step uses Adam updates and a training loop that iterates over the `tf.data` dataset for `MAX_EPOCHS`
- The epoch with the lowest combined generator-discriminator loss is saved to be recalled later

```

In [8]: # Configure CycleGAN losses and optimizers
mse = tf.keras.losses.MeanSquaredError()
optimizer_G = tf.keras.optimizers.Adam(learning_rate=2e-4, beta_1=0.5, beta_2=0.999)
optimizer_D = tf.keras.optimizers.Adam(learning_rate=2e-4, beta_1=0.5, beta_2=0.999)

# Cache variable lists for gradient application
gen_vars = monet_generator.trainable_variables + photo_generator.trainable_variables
disc_vars = monet_discriminator.trainable_variables + photo_discriminator.trainable_variables

@tf.function
# Execute a single training step
def train_step(real_photo, real_monet):
    with tf.GradientTape(persistent=True) as tape:
        fake_monet = monet_generator(real_photo, training=True)
        fake_photo = photo_generator(real_monet, training=True)

        monet_real_logits = monet_discriminator(real_monet, training=True)
        monet_fake_logits = monet_discriminator(fake_monet, training=True)
        photo_real_logits = photo_discriminator(real_photo, training=True)
        photo_fake_logits = photo_discriminator(fake_photo, training=True)

        valid = tf.ones_like(monet_real_logits)
        fake = tf.zeros_like(monet_fake_logits)

        loss_G = mse(valid, monet_fake_logits) + mse(valid, photo_fake_logits)
        loss_D_monet = mse(valid, monet_real_logits) + mse(fake, monet_fake_logits)
        loss_D_photo = mse(valid, photo_real_logits) + mse(fake, photo_fake_logits)
        loss_D = loss_D_monet + loss_D_photo

    gen_grads = tape.gradient(loss_G, gen_vars)
    disc_grads = tape.gradient(loss_D, disc_vars)

    optimizer_G.apply_gradients(zip(gen_grads, gen_vars))
    optimizer_D.apply_gradients(zip(disc_grads, disc_vars))

    return loss_G, loss_D

# Train the base Monet GAN model
def train(dataset, epochs, visualize_interval):
    global epoch_snapshots, epoch_loss_history

    steps_per_epoch = len(photo_images) // BATCH_SIZE

    # Configure best-model tracking callback
    base_selector = {
        'metric_fn': _combined_loss_metric,
        'on_best': lambda epoch_idx, loss_G, loss_D: monet_generator.save_weights(BEST_MONET_GENERATOR_PATH),
        'message_fn': lambda epoch_idx, metric_value: print(f' -> Saved best Monet generator (epoch {epoch_idx}, combined loss {metric_value:.3f})'),
    }

    # Execute shared training loop for this model
    summary = run_generator_training_session(
        session_name='Base CycleGAN',
        epochs=epochs,
        dataset_factory=lambda: dataset,
        steps_per_epoch=steps_per_epoch,
        train_step_fn=train_step,
        snapshot_generator=lambda: generate_monet_batch(comparison_photo_paths),
        snapshot_interval=visualize_interval,
        best_selector=base_selector,
    )

```



```

# Persist snapshots for downstream visualization
epoch_snapshots = summary['snapshots']
epoch_loss_history = summary['history']

# Reload best weights so downstream cells reuse them
monet_generator.load_weights(BEST_MONET_GENERATOR_PATH)
print(f'Loaded best Monet generator weights from epoch {summary['best_epoch']}")

return {
    "best_epoch": summary['best_epoch'],
    "best_combined_loss": summary['best_metric'],
    "best_generator": monet_generator,
    "history": summary['history'],
    "snapshots": summary['snapshots'],
}

# Train base CycleGAN
training_summary = train(dataset, epochs=MAX_EPOCHS, visualize_interval=1)
best_monet_generator = training_summary["best_generator"]

```

Base CycleGAN: epoch 1/20

2025-11-18 20:23:11.032851: I tensorflow/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:473] Loaded cuDNN version 91002

3500/3500 ————— 290s 81ms/step - loss_G: 0.7189 - loss_D: 0.6116

Base CycleGAN epoch 1 complete - loss_G: 0.719, loss_D: 0.612

2025-11-18 20:27:58.347572: I tensorflow/core/framework/local_rendezvous.cc:407] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence

-> Saved best Monet generator (epoch 1, combined loss 1.330)

Base CycleGAN: epoch 2/20

3500/3500 ————— 273s 78ms/step - loss_G: 1.0976 - loss_D: 0.6561

Base CycleGAN epoch 2 complete - loss_G: 1.098, loss_D: 0.656

Base CycleGAN: epoch 3/20

1/3500 ————— 5:39 97ms/step - loss_G: 0.8447 - loss_D: 0.6976

2025-11-18 20:32:32.532939: I tensorflow/core/framework/local_rendezvous.cc:407] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence

3500/3500 ————— 274s 78ms/step - loss_G: 0.7571 - loss_D: 0.8245

Base CycleGAN epoch 3 complete - loss_G: 0.757, loss_D: 0.824

Base CycleGAN: epoch 4/20

3500/3500 ————— 274s 78ms/step - loss_G: 0.7921 - loss_D: 0.4686

Base CycleGAN epoch 4 complete - loss_G: 0.792, loss_D: 0.469

-> Saved best Monet generator (epoch 4, combined loss 1.261)

Base CycleGAN: epoch 5/20

1/3500 ————— 6:01 103ms/step - loss_G: 1.3869 - loss_D: 0.3042

2025-11-18 20:41:41.407198: I tensorflow/core/framework/local_rendezvous.cc:407] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence

3500/3500 ————— 279s 80ms/step - loss_G: 0.9258 - loss_D: 0.5282

Base CycleGAN epoch 5 complete - loss_G: 0.926, loss_D: 0.528

Base CycleGAN: epoch 6/20

3500/3500 ————— 277s 79ms/step - loss_G: 0.8975 - loss_D: 0.7317

Base CycleGAN epoch 6 complete - loss_G: 0.897, loss_D: 0.732

Base CycleGAN: epoch 7/20

3500/3500 ————— 274s 78ms/step - loss_G: 0.6742 - loss_D: 0.6969

Base CycleGAN epoch 7 complete - loss_G: 0.674, loss_D: 0.697

Base CycleGAN: epoch 8/20

3500/3500 ————— 287s 82ms/step - loss_G: 0.6907 - loss_D: 0.6060

Base CycleGAN epoch 8 complete - loss_G: 0.691, loss_D: 0.606

Base CycleGAN: epoch 9/20

1/3500 ————— 5:41 97ms/step - loss_G: 1.0378 - loss_D: 0.6374

2025-11-18 21:00:19.137912: I tensorflow/core/framework/local_rendezvous.cc:407] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence

3500/3500 ————— 275s 79ms/step - loss_G: 1.4555 - loss_D: 1.3612

Base CycleGAN epoch 9 complete - loss_G: 1.456, loss_D: 1.361

Base CycleGAN: epoch 10/20

3500/3500 ————— 272s 78ms/step - loss_G: 1.6733 - loss_D: 0.3276

Base CycleGAN epoch 10 complete - loss_G: 1.673, loss_D: 0.328

Base CycleGAN: epoch 11/20

3500/3500 ————— 276s 79ms/step - loss_G: 1.2673 - loss_D: 0.4769

Base CycleGAN epoch 11 complete - loss_G: 1.267, loss_D: 0.477

Base CycleGAN: epoch 12/20

3500/3500 ————— 285s 82ms/step - loss_G: 2.1435 - loss_D: 0.3515

Base CycleGAN epoch 12 complete - loss_G: 2.143, loss_D: 0.351

Base CycleGAN: epoch 13/20

3500/3500 ————— 285s 82ms/step - loss_G: 1.3301 - loss_D: 0.4921

Base CycleGAN epoch 13 complete - loss_G: 1.330, loss_D: 0.492

Base CycleGAN: epoch 14/20

3500/3500 ————— 292s 83ms/step - loss_G: 0.8224 - loss_D: 0.5707

Base CycleGAN epoch 14 complete - loss_G: 0.822, loss_D: 0.571

Base CycleGAN: epoch 15/20

3500/3500 ————— 293s 84ms/step - loss_G: 0.8423 - loss_D: 0.6755

Base CycleGAN epoch 15 complete - loss_G: 0.842, loss_D: 0.675

Base CycleGAN: epoch 16/20

3500/3500 ————— 321s 92ms/step - loss_G: 0.8986 - loss_D: 0.5919

Base CycleGAN epoch 16 complete - loss_G: 0.899, loss_D: 0.592

Base CycleGAN: epoch 17/20

1/3500 ————— 6:05 104ms/step - loss_G: 1.0732 - loss_D: 0.6457

2025-11-18 21:38:39.771610: I tensorflow/core/framework/local_rendezvous.cc:407] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence

```

3500/3500 ————— 274s 78ms/step - loss_G: 0.6940 - loss_D: 0.9274
Base CycleGAN epoch 17 complete - loss_G: 0.694, loss_D: 0.927
Base CycleGAN: epoch 18/20
3500/3500 ————— 276s 79ms/step - loss_G: 1.1764 - loss_D: 0.4742
Base CycleGAN epoch 18 complete - loss_G: 1.176, loss_D: 0.474
Base CycleGAN: epoch 19/20
3500/3500 ————— 275s 79ms/step - loss_G: 1.0478 - loss_D: 0.5653
Base CycleGAN epoch 19 complete - loss_G: 1.048, loss_D: 0.565
Base CycleGAN: epoch 20/20
3500/3500 ————— 282s 80ms/step - loss_G: 1.3856 - loss_D: 0.5840
Base CycleGAN epoch 20 complete - loss_G: 1.386, loss_D: 0.584
Loaded best Monet generator weights from epoch 4

```

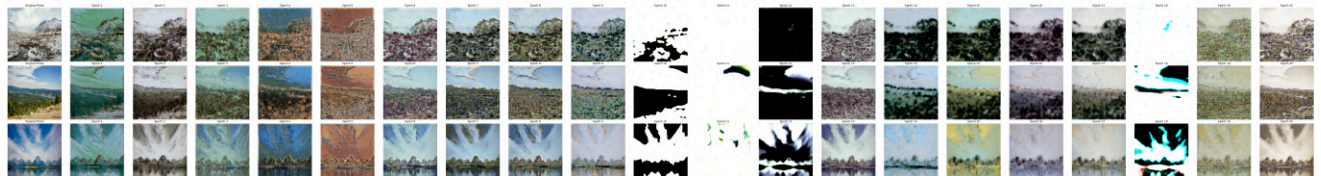
Visualize Progression by Epoch

- Shows how the training progressed over the epochs

```

In [9]: # Visualize progression over the recorded epochs
show_epoch_progression(comparison_photo_paths, training_summary["snapshots"])

```



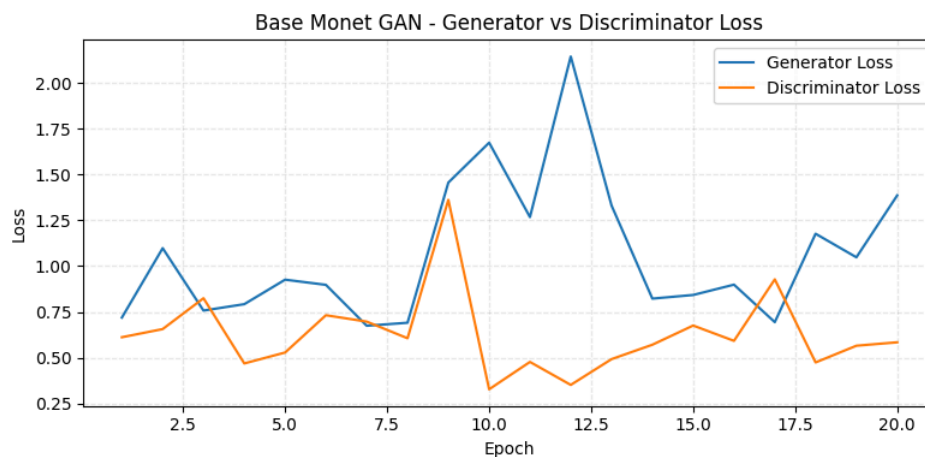
Visualize Generator-Discriminator Loss

- The plot shows how generator and discriminator losses evolve each epoch to reveal whether training stays balanced or drifts toward instability

```

In [10]: # Plot training losses for the base model
plot_loss_history(training_summary["history"], model_name="Base Monet GAN")

```



The base Monet GAN shows consistently lower discriminator losses than generator losses, with both losses peaking mid-training before settling back down. The best combined loss occurs at epoch 4, where the generator and discriminator losses are both comparatively low, marking the most balanced training point.

FID & MiFID Scores

- FID (Fréchet Inception Distance) measures how close the generated image distribution is to real Monet paintings by comparing the mean and covariance of feature embeddings from an Inception network—lower scores mean more realistic outputs.
- MiFID extends FID by also checking for memorization, penalizing generators whose outputs look too similar to specific training images so you can catch overfitting while still valuing realism.

```

In [11]: # Evaluate quantitative metrics for the base generator
base_model_metrics = calculate_fid_and_mifid(best_monet_generator, num_samples=128, batch_size=8)

print(f"\nBase Monet generator metrics:\n\
      FID: {base_model_metrics['fid']:.3f}, MiFID: {base_model_metrics['mifid']:.3f},\n\
      Memorization: {base_model_metrics['memorization']:.3f} (evaluated on {base_model_metrics['num_generated']} generated / {base_model_metrics['num_real']} real samples)")

```

2025-11-18 21:57:20.490937: W external/local_xla/xla/tsl/framework/bfc_allocator.cc:310] Allocator (GPU_0_bfc) ran out of memory trying to allocate 2.0 GiB with freed_by_count=0. The caller indicates that this is not a failure, but this may mean that there could be performance gains if more memory were available.

```

Base Monet generator metrics:
FID: 228.617, MiFID: 245.144,
Memorization: 16.527 (evaluated on 128 generated / 128 real samples)

```

Save Base Model Results

- Save model results to use later in the Analysis
- Memory is also cleared to avoid running out of it

```
In [12]: # Save base model results
base_model_record = {
    'name': 'Base CycleGAN',
    'weights_path': BEST_MONET_GENERATOR_PATH,
    'metrics': base_model_metrics,
    'epoch': training_summary['best_epoch'],
}

# Clear training_summary
training_summary["snapshots"].clear()
training_summary["history"]["generator"].clear()
training_summary["history"]["discriminator"].clear()

# Clear memory to avoid out of memory errors
best_monet_generator = None
training_summary = None
gc.collect()
```

Out[12]: 30238

Hyperparameter Tuning

Create Hyperparameter Tuning Dataset

- Dataset will exclusively be used for tuning the model

```
In [13]: # Build dataset pipeline used during tuning
def _create_dataset_for_tuning():
    ds = tf.data.Dataset.from_tensor_slices(photo_images)
    ds = ds.shuffle(buffer_size=len(photo_images), reshuffle_each_iteration=True)
    ds = ds.enumerate()
    ds = ds.map(map_pair, num_parallel_calls=AUTOTUNE)
    ds = ds.batch(BATCH_SIZE, drop_remainder=True)
    ds = ds.prefetch(AUTOTUNE)
    return ds
```

Tune Hyperparameters

- Hyperparameter tuning runs brief training trials for each learning-rate/ β setting using cloned generators and discriminators, logging the resulting generator, discriminator, and combined losses
- By ranking these short runs by lowest combined loss, we quickly identify the most promising configuration to use for the full CycleGAN training

```
In [14]: # Run quick trials over hyperparameter grid
def tune_generator_discriminator(hparam_configs, steps_per_trial=50):
    results = []
    total_steps = len(photo_images) // BATCH_SIZE
    if total_steps == 0:
        raise ValueError('Not enough images to run hyperparameter tuning.')
    steps_per_trial = max(1, min(steps_per_trial, total_steps))

    for trial_idx, config in enumerate(hparam_configs, start=1):
        print(f"Trial {trial_idx}/{len(hparam_configs)} - {config}")
        mg = build_generator(); mg.set_weights(monet_generator.get_weights())
        pg = build_generator(); pg.set_weights(photo_generator.get_weights())
        md = build_discriminator(); md.set_weights(monet_discriminator.get_weights())
        pd = build_discriminator(); pd.set_weights(photo_discriminator.get_weights())

        optimizer_G = tf.keras.optimizers.Adam(
            learning_rate=config['gen_lr'],
            beta_1=config.get('beta_1', 0.5),
            beta_2=config.get('beta_2', 0.999)
        )
        optimizer_D = tf.keras.optimizers.Adam(
            learning_rate=config['disc_lr'],
            beta_1=config.get('beta_1', 0.5),
            beta_2=config.get('beta_2', 0.999)
        )

        gen_vars_local = mg.trainable_variables + pg.trainable_variables
        disc_vars_local = md.trainable_variables + pd.trainable_variables

        @tf.function
        def tuning_step(real_photo, real_monet):
            with tf.GradientTape(persistent=True) as tape:
                fake_monet = mg(real_photo, training=True)
                fake_photo = pg(real_monet, training=True)

                monet_real_logits = md(real_monet, training=True)
                monet_fake_logits = md(fake_monet, training=True)
                photo_real_logits = pd(real_photo, training=True)
                photo_fake_logits = pd(fake_photo, training=True)

                valid = tf.ones_like(monet_real_logits)
                fake = tf.zeros_like(monet_fake_logits)

                loss_G = mse(valid, monet_fake_logits) + mse(valid, photo_fake_logits)
```

```

        loss_D_monet = mse(valid, monet_real_logits) + mse(fake, monet_fake_logits)
        loss_D_photo = mse(valid, photo_real_logits) + mse(fake, photo_fake_logits)
        loss_D = loss_D_monet + loss_D_photo

        gen_grads = tape.gradient(loss_G, gen_vars_local)
        disc_grads = tape.gradient(loss_D, disc_vars_local)

        optimizer_G.apply_gradients(zip(gen_grads, gen_vars_local))
        optimizer_D.apply_gradients(zip(disc_grads, disc_vars_local))

    return loss_G, loss_D

dataset_for_trial = _create_dataset_for_tuning()
for step, (real_photo, real_monet) in enumerate(dataset_for_trial.take(steps_per_trial), start=1):
    loss_G, loss_D = tuning_step(real_photo, real_monet)
    if step % max(1, steps_per_trial // 5) == 0 or step == steps_per_trial:
        print(f"    step {step}/{steps_per_trial} - loss_G: {float(loss_G):.4f}, loss_D: {float(loss_D):.4f}")

    combined_loss = float(loss_G + loss_D)
    trial_result = {
        'config': config,
        'generator_loss': float(loss_G),
        'discriminator_loss': float(loss_D),
        'combined_loss': combined_loss
    }
    results.append(trial_result)

results.sort(key=lambda item: item['combined_loss'])
print("\nHyperparameter tuning complete. Ranked results:")
for rank, result in enumerate(results, start=1):
    print(f"{rank}. combined={result['combined_loss']:.4f} | gen={result['generator_loss']:.4f} | disc={result['discriminator_loss']:.4f} | config")
return results

# Define hyperparameter search space
hyperparameter_grid = [
    {'gen_lr': 2e-4, 'disc_lr': 2e-4, 'beta_1': 0.5},
    {'gen_lr': 1e-4, 'disc_lr': 1e-4, 'beta_1': 0.5},
    {'gen_lr': 3e-4, 'disc_lr': 3e-4, 'beta_1': 0.5},
    {'gen_lr': 2e-4, 'disc_lr': 2e-4, 'beta_1': 0.3}
]

# Execute hyperparameter sweep to score configs
tuning_results = tune_generator_discriminator(hyperparameter_grid, steps_per_trial=50)

# Report best-performing hyperparameter config
best_hparams = tuning_results[0]['config']
print(f"\nBest hyperparameters from tuning: {best_hparams}")

```

Trial 1/4 - {'gen_lr': 0.0002, 'disc_lr': 0.0002, 'beta_1': 0.5}

step 10/50 - loss_G: 1.8032, loss_D: 0.1499
 step 20/50 - loss_G: 1.9634, loss_D: 0.1113
 step 30/50 - loss_G: 1.6504, loss_D: 0.3408
 step 40/50 - loss_G: 1.6790, loss_D: 0.1364
 step 50/50 - loss_G: 1.4876, loss_D: 0.5631

Trial 2/4 - {'gen_lr': 0.0001, 'disc_lr': 0.0001, 'beta_1': 0.5}

step 10/50 - loss_G: 1.9055, loss_D: 0.0972
 step 20/50 - loss_G: 1.8427, loss_D: 0.0992
 step 30/50 - loss_G: 1.9282, loss_D: 0.1647
 step 40/50 - loss_G: 1.8092, loss_D: 0.1223
 step 50/50 - loss_G: 1.5112, loss_D: 0.2326

Trial 3/4 - {'gen_lr': 0.0003, 'disc_lr': 0.0003, 'beta_1': 0.5}

step 10/50 - loss_G: 1.3345, loss_D: 0.2673
 step 20/50 - loss_G: 1.7459, loss_D: 0.1337
 step 30/50 - loss_G: 2.8544, loss_D: 0.7656
 step 40/50 - loss_G: 1.4923, loss_D: 0.3067
 step 50/50 - loss_G: 4.9200, loss_D: 1.6277

Trial 4/4 - {'gen_lr': 0.0002, 'disc_lr': 0.0002, 'beta_1': 0.3}

step 10/50 - loss_G: 1.9173, loss_D: 0.1561
 step 20/50 - loss_G: 1.2925, loss_D: 0.1799
 step 30/50 - loss_G: 1.5896, loss_D: 0.1532
 step 40/50 - loss_G: 1.6672, loss_D: 0.1880
 step 50/50 - loss_G: 1.8445, loss_D: 0.2221

Hyperparameter tuning complete. Ranked results:

1. combined=1.7439 | gen=1.5112 | disc=0.2326 | config={'gen_lr': 0.0001, 'disc_lr': 0.0001, 'beta_1': 0.5}
 2. combined=2.0507 | gen=1.4876 | disc=0.5631 | config={'gen_lr': 0.0002, 'disc_lr': 0.0002, 'beta_1': 0.5}
 3. combined=2.0666 | gen=1.8445 | disc=0.2221 | config={'gen_lr': 0.0002, 'disc_lr': 0.0002, 'beta_1': 0.3}
 4. combined=6.5477 | gen=4.9200 | disc=1.6277 | config={'gen_lr': 0.0003, 'disc_lr': 0.0003, 'beta_1': 0.5}

Best hyperparameters from tuning: {'gen_lr': 0.0001, 'disc_lr': 0.0001, 'beta_1': 0.5}

Train Model Using Best Hyperparameters

- Applying the best hyperparameters to the base model

```

In [15]: # Train models using the best hyperparameters
def train_model_with_best_hparams(best_config, epochs=5):

    tuned_monet_generator = build_generator()
    tuned_photo_generator = build_generator()
    tuned_monet_discriminator = build_discriminator()
    tuned_photo_discriminator = build_discriminator()

```

```

tuned_monet_generator.set_weights(monet_generator.get_weights())
tuned_photo_generator.set_weights(photo_generator.get_weights())
tuned_monet_discriminator.set_weights(monet_discriminator.get_weights())
tuned_photo_discriminator.set_weights(photo_discriminator.get_weights())

optimizer_G = tf.keras.optimizers.Adam(
    learning_rate=best_config['gen_lr'],
    beta_1=best_config.get('beta_1', 0.5),
    beta_2=best_config.get('beta_2', 0.999)
)
optimizer_D = tf.keras.optimizers.Adam(
    learning_rate=best_config['disc_lr'],
    beta_1=best_config.get('beta_1', 0.5),
    beta_2=best_config.get('beta_2', 0.999)
)

gen_vars_local = tuned_monet_generator.trainable_variables + tuned_photo_generator.trainable_variables
disc_vars_local = tuned_monet_discriminator.trainable_variables + tuned_photo_discriminator.trainable_variables

@tf.function
def tuned_train_step(real_photo, real_monet):
    with tf.GradientTape(persistent=True) as tape:
        fake_monet = tuned_monet_generator(real_photo, training=True)
        fake_photo = tuned_photo_generator(real_monet, training=True)

        monet_real_logits = tuned_monet_discriminator(real_monet, training=True)
        monet_fake_logits = tuned_monet_discriminator(fake_monet, training=True)
        photo_real_logits = tuned_photo_discriminator(real_photo, training=True)
        photo_fake_logits = tuned_photo_discriminator(fake_photo, training=True)

        valid = tf.ones_like(monet_real_logits)
        fake = tf.zeros_like(monet_fake_logits)

        loss_G = mse(valid, monet_fake_logits) + mse(valid, photo_fake_logits)
        loss_D_monet = mse(valid, monet_real_logits) + mse(fake, monet_fake_logits)
        loss_D_photo = mse(valid, photo_real_logits) + mse(fake, photo_fake_logits)
        loss_D = loss_D_monet + loss_D_photo

        gen_grads = tape.gradient(loss_G, gen_vars_local)
        disc_grads = tape.gradient(loss_D, disc_vars_local)

        optimizer_G.apply_gradients(zip(gen_grads, gen_vars_local))
        optimizer_D.apply_gradients(zip(disc_grads, disc_vars_local))

    return loss_G, loss_D

steps_per_epoch = len(photo_images) // BATCH_SIZE

def capture_best_weights(epoch_idx, loss_G, loss_D):
    return {
        'generator': tuned_monet_generator.get_weights(),
        'photo_generator': tuned_photo_generator.get_weights(),
        'monet_discriminator': tuned_monet_discriminator.get_weights(),
        'photo_discriminator': tuned_photo_discriminator.get_weights(),
    }

summary = run_generator_training_session(
    session_name='Best-hparam training',
    epochs=epochs,
    dataset_factory=_create_dataset_for_tuning,
    steps_per_epoch=steps_per_epoch,
    train_step_fn=tuned_train_step,
    snapshot_generator=lambda: generate_monet_batch(comparison_photo_paths, generator=tuned_monet_generator),
    snapshot_interval=1,
    best_selector={
        'metric_fn': _difference_loss_metric,
        'on_best': capture_best_weights,
        'message_fn': lambda epoch_idx, metric_value: print(f" -> Stored best tuned weights from epoch {epoch_idx} (|G-D| diff {metric_value:.4f})"),
    },
)

history = summary['history']
tuned_epoch_snapshots = summary['snapshots']
best_weights = summary['best_payload']
best_epoch = summary['best_epoch']
best_diff = summary['best_metric']

tuned_monet_generator.set_weights(best_weights['generator'])
tuned_photo_generator.set_weights(best_weights['photo_generator'])
tuned_monet_discriminator.set_weights(best_weights['monet_discriminator'])
tuned_photo_discriminator.set_weights(best_weights['photo_discriminator'])
print(f"Loaded best tuned weights from epoch {best_epoch} with |G-D| diff {best_diff:.4f}")
best_weights = None
print('Finished training copy of the model with best hyperparameters.\n')

# Visualize epochs & model loss
show_epoch_progression(comparison_photo_paths, tuned_epoch_snapshots)
plot_loss_history(history, model_name='Best Hyperparameter Model')

# Clear memory
tuned_epoch_snapshots.clear()
history['generator'].clear()
history['discriminator'].clear()

```



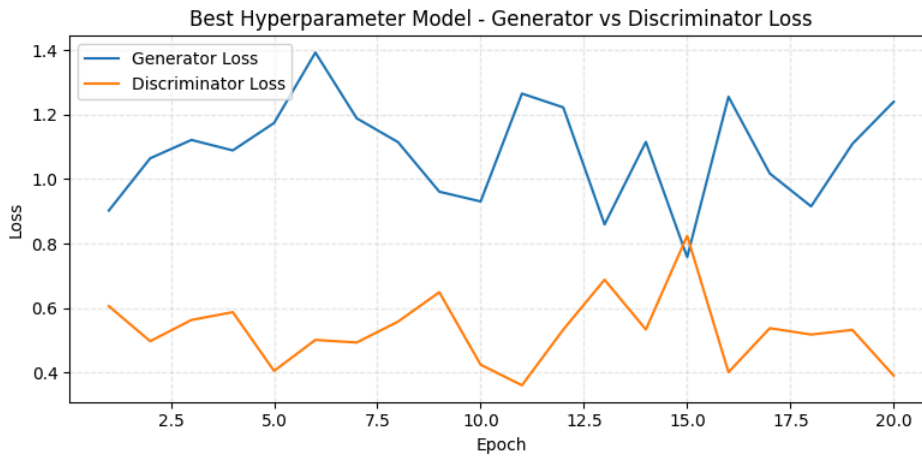
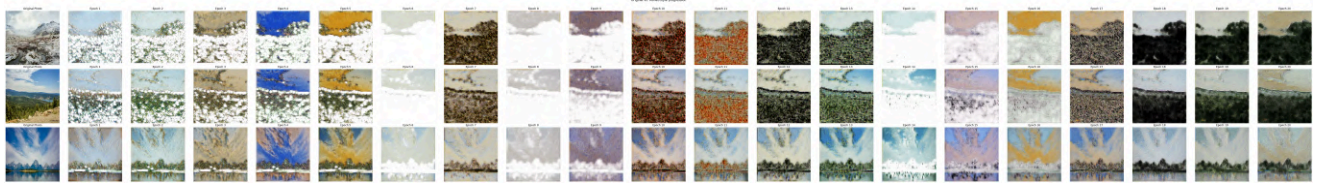
```

return {
    'generator': tuned_monet_generator,
    'photo_generator': tuned_photo_generator,
    'monet_discriminator': tuned_monet_discriminator,
    'photo_discriminator': tuned_photo_discriminator,
    'best_epoch': best_epoch,
    'best_diff': best_diff,
    'best_weights': best_weights,
    'history': history,
    'snapshots': tuned_epoch_snapshots,
    'config': best_config,
}

# Train tuned model if we found a winning config
best_hparam_training = train_model_with_best_hparams(best_hparams, epochs=MAX_EPOCHS)

Best-hparam training: epoch 1/20
3500/3500 ————— 280s 79ms/step - loss_G: 0.9026 - loss_D: 0.6058
Best-hparam training epoch 1 complete - loss_G: 0.903, loss_D: 0.606
-> Stored best tuned weights from epoch 1 (|G-D| diff 0.2968)
Best-hparam training: epoch 2/20
3500/3500 ————— 272s 78ms/step - loss_G: 1.0648 - loss_D: 0.4969
Best-hparam training epoch 2 complete - loss_G: 1.065, loss_D: 0.497
Best-hparam training: epoch 3/20
3500/3500 ————— 273s 78ms/step - loss_G: 1.1219 - loss_D: 0.5628
Best-hparam training epoch 3 complete - loss_G: 1.122, loss_D: 0.563
Best-hparam training: epoch 4/20
3500/3500 ————— 275s 79ms/step - loss_G: 1.0893 - loss_D: 0.5871
Best-hparam training epoch 4 complete - loss_G: 1.089, loss_D: 0.587
Best-hparam training: epoch 5/20
3500/3500 ————— 273s 78ms/step - loss_G: 1.1746 - loss_D: 0.4047
Best-hparam training epoch 5 complete - loss_G: 1.175, loss_D: 0.405
Best-hparam training: epoch 6/20
3500/3500 ————— 278s 79ms/step - loss_G: 1.3933 - loss_D: 0.5009
Best-hparam training epoch 6 complete - loss_G: 1.393, loss_D: 0.501
Best-hparam training: epoch 7/20
3500/3500 ————— 276s 79ms/step - loss_G: 1.1891 - loss_D: 0.4928
Best-hparam training epoch 7 complete - loss_G: 1.189, loss_D: 0.493
Best-hparam training: epoch 8/20
3500/3500 ————— 275s 79ms/step - loss_G: 1.1151 - loss_D: 0.5577
Best-hparam training epoch 8 complete - loss_G: 1.115, loss_D: 0.558
Best-hparam training: epoch 9/20
1/3500 ————— 5:43 98ms/step - loss_G: 1.2769 - loss_D: 0.6278
2025-11-18 22:37:44.118720: I tensorflow/core/framework/local_rendezvous.cc:407] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
e
3500/3500 ————— 273s 78ms/step - loss_G: 0.9611 - loss_D: 0.6489
Best-hparam training epoch 9 complete - loss_G: 0.961, loss_D: 0.649
Best-hparam training: epoch 10/20
3500/3500 ————— 273s 78ms/step - loss_G: 0.9308 - loss_D: 0.4239
Best-hparam training epoch 10 complete - loss_G: 0.931, loss_D: 0.424
Best-hparam training: epoch 11/20
3500/3500 ————— 274s 78ms/step - loss_G: 1.2658 - loss_D: 0.3600
Best-hparam training epoch 11 complete - loss_G: 1.266, loss_D: 0.360
Best-hparam training: epoch 12/20
3500/3500 ————— 275s 79ms/step - loss_G: 1.2229 - loss_D: 0.5328
Best-hparam training epoch 12 complete - loss_G: 1.223, loss_D: 0.533
Best-hparam training: epoch 13/20
3500/3500 ————— 275s 79ms/step - loss_G: 0.8595 - loss_D: 0.6878
Best-hparam training epoch 13 complete - loss_G: 0.860, loss_D: 0.688
-> Stored best tuned weights from epoch 13 (|G-D| diff 0.1717)
Best-hparam training: epoch 14/20
3500/3500 ————— 276s 79ms/step - loss_G: 1.1157 - loss_D: 0.5332
Best-hparam training epoch 14 complete - loss_G: 1.116, loss_D: 0.533
Best-hparam training: epoch 15/20
3500/3500 ————— 274s 78ms/step - loss_G: 0.7580 - loss_D: 0.8242
Best-hparam training epoch 15 complete - loss_G: 0.758, loss_D: 0.824
-> Stored best tuned weights from epoch 15 (|G-D| diff 0.0662)
Best-hparam training: epoch 16/20
3500/3500 ————— 280s 80ms/step - loss_G: 1.2560 - loss_D: 0.4006
Best-hparam training epoch 16 complete - loss_G: 1.256, loss_D: 0.401
Best-hparam training: epoch 17/20
3500/3500 ————— 279s 80ms/step - loss_G: 1.0177 - loss_D: 0.5373
Best-hparam training epoch 17 complete - loss_G: 1.018, loss_D: 0.537
Best-hparam training: epoch 18/20
3500/3500 ————— 280s 80ms/step - loss_G: 0.9155 - loss_D: 0.5175
Best-hparam training epoch 18 complete - loss_G: 0.916, loss_D: 0.518
Best-hparam training: epoch 19/20
3500/3500 ————— 282s 81ms/step - loss_G: 1.1097 - loss_D: 0.5320
Best-hparam training epoch 19 complete - loss_G: 1.110, loss_D: 0.532
Best-hparam training: epoch 20/20
3500/3500 ————— 281s 80ms/step - loss_G: 1.2406 - loss_D: 0.3898
Best-hparam training epoch 20 complete - loss_G: 1.241, loss_D: 0.390
Loaded best tuned weights from epoch 15 with |G-D| diff 0.0662
Finished training copy of the model with best hyperparameters.

```



The tuned CycleGAN keeps discriminator loss consistently below generator loss while both oscillate as the model searches for balance, with the sharpest alignment—and lowest combined loss—arriving at epoch 15. This suggests the hyperparameter sweep produced a configuration that briefly harmonizes the adversaries before generator loss drifts higher again.

Best Hyperparameter Tuned FID & MiFID Scores

```
In [16]: # Compute metrics for the tuned GAN model
tuned_metrics = calculate_fid_and_mifid(best_hparam_training['generator'], num_samples=128, batch_size=8)
print(f'Tuned Monet generator metrics:\n'
      f'FID: {tuned_metrics['fid']:.3f}, MiFID: {tuned_metrics['mifid']:.3f},\n'
      f'Memorization: {tuned_metrics['memorization']:.3f} (evaluated on {tuned_metrics['num_generated']} generated / {tuned_metrics['num_real']} real samp
```

Tuned Monet generator metrics:
 FID: 237.703, MiFID: 253.673,
 Memorization: 15.970 (evaluated on 128 generated / 128 real samples)

Save Model & Clear Memory

```
In [17]: # Save model
tuned_weights_path = os.path.join(CHECKPOINT_DIR, 'tuned_generator_best.weights.h5')
best_hparam_training['generator'].save_weights(tuned_weights_path)
tuned_model_record = {
    'name': 'Tuned Monet GAN',
    'weights_path': tuned_weights_path,
    'metrics': tuned_metrics,
    'epoch': best_hparam_training['best_epoch'],
}

# Clear memory
best_hparam_training['generator'] = None
best_hparam_training['photo_generator'] = None
best_hparam_training['monet_discriminator'] = None
best_hparam_training['photo_discriminator'] = None
best_hparam_training['snapshots'] = {}
best_hparam_training['history'] = {'generator': [], 'discriminator': []}
gc.collect()
```

Out[17]: 193834

GAN Model Variants

Three GAN model variants are created for comparison: Least Squares GAN, Hinge GAN, and Wasserstein GAN.

- **Least Squares GAN** simply swaps the adversarial loss with an L2 objective to stabilize training
- **Hinge GAN** replaces the loss with hinge margins that penalize only when discriminators mis-rank real/fake samples
- **Wasserstein GAN** measures critic scores with the Earth-Mover distance and enforces smoother gradients through different optimizer betas

All three keep the same CycleGAN topology (two generators + two discriminators with cycle-consistency), differing only in how the adversarial losses shape generator–discriminator updates.

Train Variant Models & Display Results

- Similar outputs to the base and tuned models are also created for each GAN variant

```

In [18]: # Compute generator and discriminator losses for a variant
def _compute_variant_losses(loss_type, real_logits, fake_logits):
    if loss_type == 'least_squares':
        valid = tf.ones_like(real_logits)
        fake = tf.zeros_like(fake_logits)
        disc_loss = tf.reduce_mean(tf.square(real_logits - valid)) + tf.reduce_mean(tf.square(fake_logits - fake))
        gen_loss = tf.reduce_mean(tf.square(fake_logits - valid))
    elif loss_type == 'hinge':
        disc_loss = tf.reduce_mean(tf.nn.relu(1.0 - real_logits)) + tf.reduce_mean(tf.nn.relu(1.0 + fake_logits))
        gen_loss = -tf.reduce_mean(fake_logits)
    elif loss_type == 'wasserstein':
        disc_loss = tf.reduce_mean(fake_logits) - tf.reduce_mean(real_logits)
        gen_loss = -tf.reduce_mean(fake_logits)
    else:
        raise ValueError(f'Unsupported loss type: {loss_type}')
    return gen_loss, disc_loss

# Evaluate multiple GAN loss variants with shared tooling
def test_gan_variants(variant_configs, epochs=3):
    steps_per_epoch = len(photo_images) // BATCH_SIZE
    results = []

    for cfg in variant_configs:
        variant_name = cfg.get('name', cfg.get('loss_type', 'variant'))
        loss_type = cfg.get('loss_type', 'least_squares')
        gen_lr = cfg.get('gen_lr', 2e-4)
        disc_lr = cfg.get('disc_lr', 2e-4)
        beta_1 = cfg.get('beta_1', 0.5)
        beta_2 = cfg.get('beta_2', 0.999)

        print(f'\n== Training variant: {variant_name} ({loss_type}) ==')
        mg = build_generator(); mg.set_weights(monet_generator.get_weights())
        pg = build_generator(); pg.set_weights(photo_generator.get_weights())
        md = build_discriminator(); md.set_weights(monet_discriminator.get_weights())
        pd = build_discriminator(); pd.set_weights(photo_discriminator.get_weights())

        optimizer_G = tf.keras.optimizers.Adam(learning_rate=gen_lr, beta_1=beta_1, beta_2=beta_2)
        optimizer_D = tf.keras.optimizers.Adam(learning_rate=disc_lr, beta_1=beta_1, beta_2=beta_2)

        gen_vars_local = mg.trainable_variables + pg.trainable_variables
        disc_vars_local = md.trainable_variables + pd.trainable_variables

        @tf.function
        def variant_train_step(real_photo, real_monet):
            with tf.GradientTape(persistent=True) as tape:
                fake_monet = mg(real_photo, training=True)
                fake_photo = pg(real_monet, training=True)

                monet_real_logits = md(real_monet, training=True)
                monet_fake_logits = md(fake_monet, training=True)
                photo_real_logits = pd(real_photo, training=True)
                photo_fake_logits = pd(fake_photo, training=True)

                gen_loss_monet, disc_loss_monet = _compute_variant_losses(loss_type, monet_real_logits, monet_fake_logits)
                gen_loss_photo, disc_loss_photo = _compute_variant_losses(loss_type, photo_real_logits, photo_fake_logits)

                loss_G = gen_loss_monet + gen_loss_photo
                loss_D = disc_loss_monet + disc_loss_photo

            gen_grads = tape.gradient(loss_G, gen_vars_local)
            disc_grads = tape.gradient(loss_D, disc_vars_local)

            optimizer_G.apply_gradients(zip(gen_grads, gen_vars_local))
            optimizer_D.apply_gradients(zip(disc_grads, disc_vars_local))

            return loss_G, loss_D

        def capture_variant_weights(epoch_idx, loss_G, loss_D):
            return {
                'generator': mg.get_weights(),
                'photo_generator': pg.get_weights(),
                'monet_discriminator': md.get_weights(),
                'photo_discriminator': pd.get_weights(),
            }

        summary = run_generator_training_session(
            session_name=f'GAN variant: {variant_name}',
            epochs=epochs,
            dataset_factory=create_dataset_for_tuning,
            steps_per_epoch=steps_per_epoch,
            train_step_fn=variant_train_step,
            snapshot_generator=lambda: generate_monet_batch(comparison_photo_paths, generator=mg),
            snapshot_interval=1,
            best_selector={
                'metric_fn': _difference_loss_metric,
                'on_best': capture_variant_weights,
                'message_fn': lambda epoch_idx, metric_value, name=variant_name: print(f' -> Stored best {name} weights (|G-D| diff {metric_value:.4f}
            ),
        )

        history = summary['history']
        tuned_epoch_snapshots = summary['snapshots']
        best_weights = summary['best_payload']

```

```

best_epoch = summary['best_epoch']
best_diff = summary['best_metric']

if best_weights is not None:
    mg.set_weights(best_weights['generator'])
    pg.set_weights(best_weights['photo_generator'])
    md.set_weights(best_weights['monet_discriminator'])
    pd.set_weights(best_weights['photo_discriminator'])
    print(f'\nLoaded best {variant_name} weights from epoch {best_epoch} with |G-D| diff {best_diff:.4f}')
best_weights = None

# Visualize epoch progression & model loss
show_epoch_progression(comparison_photo_paths, tuned_epoch_snapshots)
plot_loss_history(history, model_name=variant_name)

# Calculate metrics & save model weights
variant_metrics = calculate_fid_and_mifid(mg, num_samples=128, batch_size=8)
variant_weights_name = variant_name.lower().replace(' ', '_').replace('/', '_')
variant_weights_path = os.path.join(CHECKPOINT_DIR, f'{variant_weights_name}_generator.weights.h5')
mg.save_weights(variant_weights_path)

# Clear memory
tuned_epoch_snapshots.clear()
history['generator'].clear()
history['discriminator'].clear()

# Display FID & MiFID metrics
print(f'\n{variant_name} metrics:\n\
FID: {variant_metrics['fid']:.3f}, MiFID: {variant_metrics['mifid']:.3f},\n\
Memorization: {variant_metrics['memorization']:.3f}')

results.append({
    'name': variant_name,
    'loss_type': loss_type,
    'history': history,
    'snapshots': tuned_epoch_snapshots,
    'metrics': variant_metrics,
    'weights_path': variant_weights_path,
    'best_epoch': best_epoch,
    'best_diff': best_diff,
    'generator': None,
    'photo_generator': None,
    'monet_discriminator': None,
    'photo_discriminator': None,
})

del mg, pg, md, pd
gc.collect()
return results

# Define GAN variants
gan_variant_configs = [
    {'name': 'Least Squares GAN', 'loss_type': 'least_squares', 'gen_lr': 2e-4, 'disc_lr': 2e-4},
    {'name': 'Hinge GAN', 'loss_type': 'hinge', 'gen_lr': 2e-4, 'disc_lr': 2e-4},
    {'name': 'Wasserstein GAN', 'loss_type': 'wasserstein', 'gen_lr': 1e-4, 'disc_lr': 1e-4, 'beta_1': 0.0, 'beta_2': 0.9}
]

# Run experiments for each configured GAN variant
gan_variant_results = test_gan_variants(gan_variant_configs, epochs=MAX_EPOCHS)

```

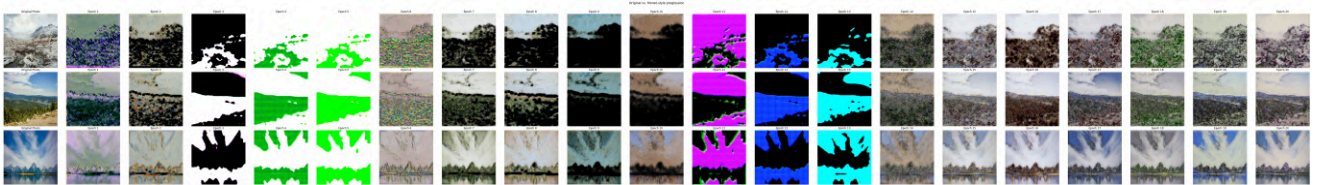
```

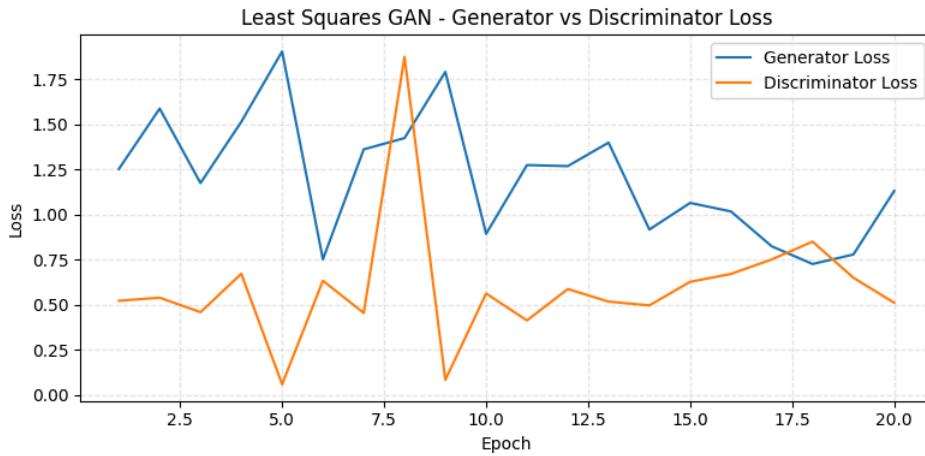
=== Training variant: Least Squares GAN (least_squares) ===
GAN variant: Least Squares GAN: epoch 1/20
3500/3500 ----- 275s 78ms/step - loss_G: 1.2529 - loss_D: 0.5213
GAN variant: Least Squares GAN epoch 1 complete - loss_G: 1.253, loss_D: 0.521
-> Stored best Least Squares GAN weights (|G-D| diff 0.7316)
GAN variant: Least Squares GAN: epoch 2/20
3500/3500 ----- 279s 80ms/step - loss_G: 1.5884 - loss_D: 0.5385
GAN variant: Least Squares GAN epoch 2 complete - loss_G: 1.588, loss_D: 0.539
GAN variant: Least Squares GAN: epoch 3/20
3500/3500 ----- 285s 81ms/step - loss_G: 1.1746 - loss_D: 0.4580
GAN variant: Least Squares GAN epoch 3 complete - loss_G: 1.175, loss_D: 0.458
-> Stored best Least Squares GAN weights (|G-D| diff 0.7166)
GAN variant: Least Squares GAN: epoch 4/20
3500/3500 ----- 275s 78ms/step - loss_G: 1.5149 - loss_D: 0.6712
GAN variant: Least Squares GAN epoch 4 complete - loss_G: 1.515, loss_D: 0.671
GAN variant: Least Squares GAN: epoch 5/20
3500/3500 ----- 279s 80ms/step - loss_G: 1.9050 - loss_D: 0.0577
GAN variant: Least Squares GAN epoch 5 complete - loss_G: 1.905, loss_D: 0.058
GAN variant: Least Squares GAN: epoch 6/20
3500/3500 ----- 280s 80ms/step - loss_G: 0.7514 - loss_D: 0.6326
GAN variant: Least Squares GAN epoch 6 complete - loss_G: 0.751, loss_D: 0.633
-> Stored best Least Squares GAN weights (|G-D| diff 0.1189)
GAN variant: Least Squares GAN: epoch 7/20
3500/3500 ----- 277s 79ms/step - loss_G: 1.3618 - loss_D: 0.4537
GAN variant: Least Squares GAN epoch 7 complete - loss_G: 1.362, loss_D: 0.454
GAN variant: Least Squares GAN: epoch 8/20
3500/3500 ----- 276s 79ms/step - loss_G: 1.4245 - loss_D: 1.8760
GAN variant: Least Squares GAN epoch 8 complete - loss_G: 1.424, loss_D: 1.876
GAN variant: Least Squares GAN: epoch 9/20
3500/3500 ----- 276s 79ms/step - loss_G: 1.7920 - loss_D: 0.0823
GAN variant: Least Squares GAN epoch 9 complete - loss_G: 1.792, loss_D: 0.082
GAN variant: Least Squares GAN: epoch 10/20
3500/3500 ----- 284s 81ms/step - loss_G: 0.8929 - loss_D: 0.5620
GAN variant: Least Squares GAN epoch 10 complete - loss_G: 0.893, loss_D: 0.562
GAN variant: Least Squares GAN: epoch 11/20
3500/3500 ----- 298s 85ms/step - loss_G: 1.2746 - loss_D: 0.4121
GAN variant: Least Squares GAN epoch 11 complete - loss_G: 1.275, loss_D: 0.412
GAN variant: Least Squares GAN: epoch 12/20
3500/3500 ----- 295s 84ms/step - loss_G: 1.2688 - loss_D: 0.5864
GAN variant: Least Squares GAN epoch 12 complete - loss_G: 1.269, loss_D: 0.586
GAN variant: Least Squares GAN: epoch 13/20
3500/3500 ----- 294s 84ms/step - loss_G: 1.3994 - loss_D: 0.5167
GAN variant: Least Squares GAN epoch 13 complete - loss_G: 1.399, loss_D: 0.517
GAN variant: Least Squares GAN: epoch 14/20
3500/3500 ----- 304s 87ms/step - loss_G: 0.9168 - loss_D: 0.4957
GAN variant: Least Squares GAN epoch 14 complete - loss_G: 0.917, loss_D: 0.496
GAN variant: Least Squares GAN: epoch 15/20
3500/3500 ----- 315s 90ms/step - loss_G: 1.0647 - loss_D: 0.6267
GAN variant: Least Squares GAN epoch 15 complete - loss_G: 1.065, loss_D: 0.627
GAN variant: Least Squares GAN: epoch 16/20
3500/3500 ----- 312s 89ms/step - loss_G: 1.0171 - loss_D: 0.6703
GAN variant: Least Squares GAN epoch 16 complete - loss_G: 1.017, loss_D: 0.670
GAN variant: Least Squares GAN: epoch 17/20
3500/3500 ----- 308s 88ms/step - loss_G: 0.8235 - loss_D: 0.7511
GAN variant: Least Squares GAN epoch 17 complete - loss_G: 0.823, loss_D: 0.751
-> Stored best Least Squares GAN weights (|G-D| diff 0.0724)
GAN variant: Least Squares GAN: epoch 18/20
3500/3500 ----- 312s 89ms/step - loss_G: 0.7254 - loss_D: 0.8495
GAN variant: Least Squares GAN epoch 18 complete - loss_G: 0.725, loss_D: 0.850
GAN variant: Least Squares GAN: epoch 19/20
3500/3500 ----- 311s 89ms/step - loss_G: 0.7783 - loss_D: 0.6493
GAN variant: Least Squares GAN epoch 19 complete - loss_G: 0.778, loss_D: 0.649
GAN variant: Least Squares GAN: epoch 20/20
3500/3500 ----- 316s 90ms/step - loss_G: 1.1313 - loss_D: 0.5102
GAN variant: Least Squares GAN epoch 20 complete - loss_G: 1.131, loss_D: 0.510

```

Loaded best Least Squares GAN weights from epoch 17 with |G-D| diff 0.0724

2025-11-19 01:14:51.682759: I tensorflow/core/framework/local_rendezvous.cc:407] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence





Least Squares GAN metrics:

FID: 176.801, MiFID: 192.228,
Memorization: 15.427

=== Training variant: Hinge GAN (hinge) ===

GAN variant: Hinge GAN: epoch 1/20

3500/3500 273s 77ms/step - loss_G: 0.8532 - loss_D: 3.1273

GAN variant: Hinge GAN epoch 1 complete - loss_G: 0.853, loss_D: 3.127

-> Stored best Hinge GAN weights (|G-D| diff 2.2741)

GAN variant: Hinge GAN: epoch 2/20

3500/3500 268s 77ms/step - loss_G: 2.0512 - loss_D: 2.0871

GAN variant: Hinge GAN epoch 2 complete - loss_G: 2.051, loss_D: 2.087

-> Stored best Hinge GAN weights (|G-D| diff 0.0358)

GAN variant: Hinge GAN: epoch 3/20

3500/3500 274s 78ms/step - loss_G: 3.4791 - loss_D: 2.2412

GAN variant: Hinge GAN epoch 3 complete - loss_G: 3.479, loss_D: 2.241

GAN variant: Hinge GAN: epoch 4/20

3500/3500 276s 79ms/step - loss_G: 0.2219 - loss_D: 3.0501

GAN variant: Hinge GAN epoch 4 complete - loss_G: 0.222, loss_D: 3.050

GAN variant: Hinge GAN: epoch 5/20

3500/3500 276s 79ms/step - loss_G: 4.8483 - loss_D: 1.3020

GAN variant: Hinge GAN epoch 5 complete - loss_G: 4.848, loss_D: 1.302

GAN variant: Hinge GAN: epoch 6/20

3500/3500 277s 79ms/step - loss_G: 2.7310 - loss_D: 1.5475

GAN variant: Hinge GAN epoch 6 complete - loss_G: 2.731, loss_D: 1.548

GAN variant: Hinge GAN: epoch 7/20

3500/3500 278s 79ms/step - loss_G: 3.7310 - loss_D: 0.5845

GAN variant: Hinge GAN epoch 7 complete - loss_G: 3.731, loss_D: 0.585

GAN variant: Hinge GAN: epoch 8/20

3500/3500 278s 80ms/step - loss_G: 3.4228 - loss_D: 1.1115

GAN variant: Hinge GAN epoch 8 complete - loss_G: 3.423, loss_D: 1.112

GAN variant: Hinge GAN: epoch 9/20

3500/3500 281s 80ms/step - loss_G: 2.3694 - loss_D: 1.5741

GAN variant: Hinge GAN epoch 9 complete - loss_G: 2.369, loss_D: 1.574

GAN variant: Hinge GAN: epoch 10/20

3500/3500 282s 81ms/step - loss_G: 2.0840 - loss_D: 1.7574

GAN variant: Hinge GAN epoch 10 complete - loss_G: 2.084, loss_D: 1.757

GAN variant: Hinge GAN: epoch 11/20

3500/3500 279s 80ms/step - loss_G: 2.1524 - loss_D: 1.8824

GAN variant: Hinge GAN epoch 11 complete - loss_G: 2.152, loss_D: 1.882

GAN variant: Hinge GAN: epoch 12/20

3500/3500 279s 80ms/step - loss_G: 2.7955 - loss_D: 2.8358

GAN variant: Hinge GAN epoch 12 complete - loss_G: 2.796, loss_D: 2.836

GAN variant: Hinge GAN: epoch 13/20

3500/3500 280s 80ms/step - loss_G: 2.2471 - loss_D: 1.1921

GAN variant: Hinge GAN epoch 13 complete - loss_G: 2.247, loss_D: 1.192

GAN variant: Hinge GAN: epoch 14/20

3500/3500 280s 80ms/step - loss_G: 4.1649 - loss_D: 2.7759

GAN variant: Hinge GAN epoch 14 complete - loss_G: 4.165, loss_D: 2.776

GAN variant: Hinge GAN: epoch 15/20

3500/3500 281s 80ms/step - loss_G: 1.6931 - loss_D: 2.7209

GAN variant: Hinge GAN epoch 15 complete - loss_G: 1.693, loss_D: 2.721

GAN variant: Hinge GAN: epoch 16/20

3500/3500 281s 80ms/step - loss_G: 2.9302 - loss_D: 2.1365

GAN variant: Hinge GAN epoch 16 complete - loss_G: 2.930, loss_D: 2.137

GAN variant: Hinge GAN: epoch 17/20

3500/3500 281s 80ms/step - loss_G: 3.7111 - loss_D: 1.5879

GAN variant: Hinge GAN epoch 17 complete - loss_G: 3.711, loss_D: 1.588

GAN variant: Hinge GAN: epoch 18/20

3500/3500 282s 80ms/step - loss_G: -0.0861 - loss_D: 3.6867

GAN variant: Hinge GAN epoch 18 complete - loss_G: -0.086, loss_D: 3.687

GAN variant: Hinge GAN: epoch 19/20

3500/3500 282s 81ms/step - loss_G: 3.6990 - loss_D: 2.0000

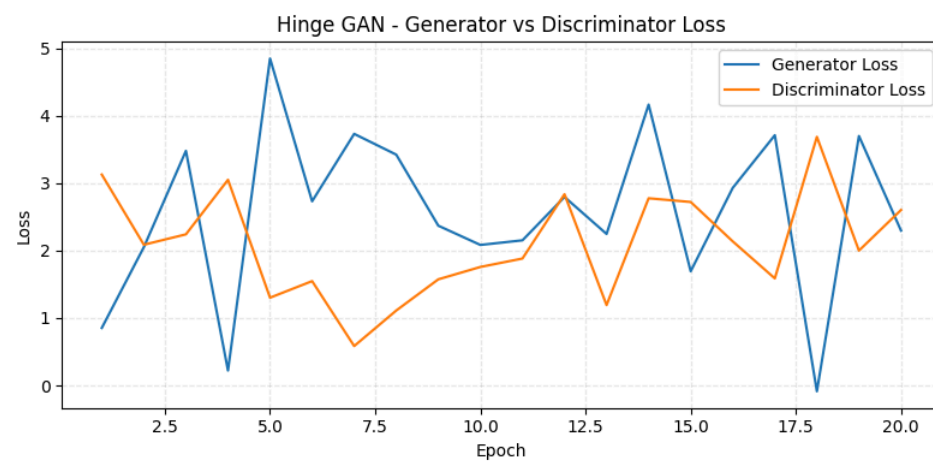
GAN variant: Hinge GAN epoch 19 complete - loss_G: 3.699, loss_D: 2.000

GAN variant: Hinge GAN: epoch 20/20

3500/3500 283s 81ms/step - loss_G: 2.2966 - loss_D: 2.6024

GAN variant: Hinge GAN epoch 20 complete - loss_G: 2.297, loss_D: 2.602

Loaded best Hinge GAN weights from epoch 2 with |G-D| diff 0.0358



Hinge GAN metrics:

FID: 276.733, MiFID: 293.202,
Memorization: 16.469

=== Training variant: Wasserstein GAN (wasserstein) ===

GAN variant: Wasserstein GAN: epoch 1/20

3500/3500 282s 80ms/step - loss_G: -233.8506 - loss_D: -107.9385

GAN variant: Wasserstein GAN epoch 1 complete - loss_G: -233.851, loss_D: -107.939

-> Stored best Wasserstein GAN weights (|G-D| diff 125.9121)

GAN variant: Wasserstein GAN: epoch 2/20

3500/3500 280s 80ms/step - loss_G: 1284.6421 - loss_D: -758.7452

GAN variant: Wasserstein GAN epoch 2 complete - loss_G: 1284.642, loss_D: -758.745

GAN variant: Wasserstein GAN: epoch 3/20

3500/3500 289s 83ms/step - loss_G: -1058.0288 - loss_D: 459.4037

GAN variant: Wasserstein GAN epoch 3 complete - loss_G: -1058.029, loss_D: 459.404

GAN variant: Wasserstein GAN: epoch 4/20

3500/3500 315s 90ms/step - loss_G: 328.8678 - loss_D: -1170.9463

GAN variant: Wasserstein GAN epoch 4 complete - loss_G: 328.868, loss_D: -1170.946

GAN variant: Wasserstein GAN: epoch 5/20

3500/3500 319s 91ms/step - loss_G: 1673.5813 - loss_D: 54.3774

GAN variant: Wasserstein GAN epoch 5 complete - loss_G: 1673.581, loss_D: 54.377

GAN variant: Wasserstein GAN: epoch 6/20

3500/3500 330s 94ms/step - loss_G: -5692.2910 - loss_D: 9163.5664

GAN variant: Wasserstein GAN epoch 6 complete - loss_G: -5692.291, loss_D: 9163.566

GAN variant: Wasserstein GAN: epoch 7/20

3500/3500 333s 95ms/step - loss_G: 3141.9641 - loss_D: -83.5289

GAN variant: Wasserstein GAN epoch 7 complete - loss_G: 3141.964, loss_D: -83.529

GAN variant: Wasserstein GAN: epoch 8/20

3500/3500 337s 96ms/step - loss_G: 1433.8979 - loss_D: -1533.0127

GAN variant: Wasserstein GAN epoch 8 complete - loss_G: 1433.898, loss_D: -1533.013

GAN variant: Wasserstein GAN: epoch 9/20

3500/3500 338s 97ms/step - loss_G: 2373.7141 - loss_D: 990.8447

GAN variant: Wasserstein GAN epoch 9 complete - loss_G: 2373.714, loss_D: 990.845

GAN variant: Wasserstein GAN: epoch 10/20

3500/3500 340s 97ms/step - loss_G: -17194.2148 - loss_D: 18427.0918

GAN variant: Wasserstein GAN epoch 10 complete - loss_G: -17194.215, loss_D: 18427.092

GAN variant: Wasserstein GAN: epoch 11/20

3500/3500 343s 98ms/step - loss_G: 4577.7065 - loss_D: -6021.9717

GAN variant: Wasserstein GAN epoch 11 complete - loss_G: 4577.707, loss_D: -6021.972

GAN variant: Wasserstein GAN: epoch 12/20

3500/3500 344s 98ms/step - loss_G: 42290.9141 - loss_D: -28567.2383

GAN variant: Wasserstein GAN epoch 12 complete - loss_G: 42290.914, loss_D: -28567.238

GAN variant: Wasserstein GAN: epoch 13/20

3500/3500 352s 100ms/step - loss_G: 51461.5234 - loss_D: -14835.8457

GAN variant: Wasserstein GAN epoch 13 complete - loss_G: 51461.523, loss_D: -14835.846

GAN variant: Wasserstein GAN: epoch 14/20

3500/3500 350s 100ms/step - loss_G: -12803.5215 - loss_D: 27828.8105

GAN variant: Wasserstein GAN epoch 14 complete - loss_G: -12803.521, loss_D: 27828.811

GAN variant: Wasserstein GAN: epoch 15/20

3500/3500 352s 101ms/step - loss_G: 39843.6055 - loss_D: 14493.5107

GAN variant: Wasserstein GAN epoch 15 complete - loss_G: 39843.605, loss_D: 14493.511

GAN variant: Wasserstein GAN: epoch 16/20

3500/3500 356s 102ms/step - loss_G: 5345.6348 - loss_D: 19367.0371

GAN variant: Wasserstein GAN epoch 16 complete - loss_G: 5345.635, loss_D: 19367.037

GAN variant: Wasserstein GAN: epoch 17/20

3500/3500 357s 102ms/step - loss_G: 73215.8750 - loss_D: -50433.3203

GAN variant: Wasserstein GAN epoch 17 complete - loss_G: 73215.875, loss_D: -50433.320

GAN variant: Wasserstein GAN: epoch 18/20

3500/3500 360s 103ms/step - loss_G: 25033.4961 - loss_D: 13401.9316

GAN variant: Wasserstein GAN epoch 18 complete - loss_G: 25033.496, loss_D: 13401.932

GAN variant: Wasserstein GAN: epoch 19/20

3500/3500 362s 104ms/step - loss_G: 27105.2129 - loss_D: -4967.8931

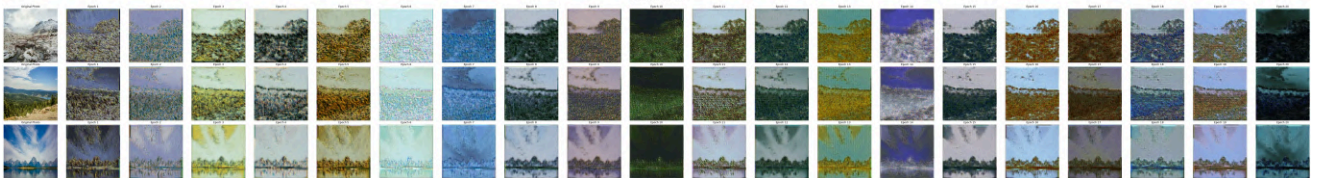
GAN variant: Wasserstein GAN epoch 19 complete - loss_G: 27105.213, loss_D: -4967.893

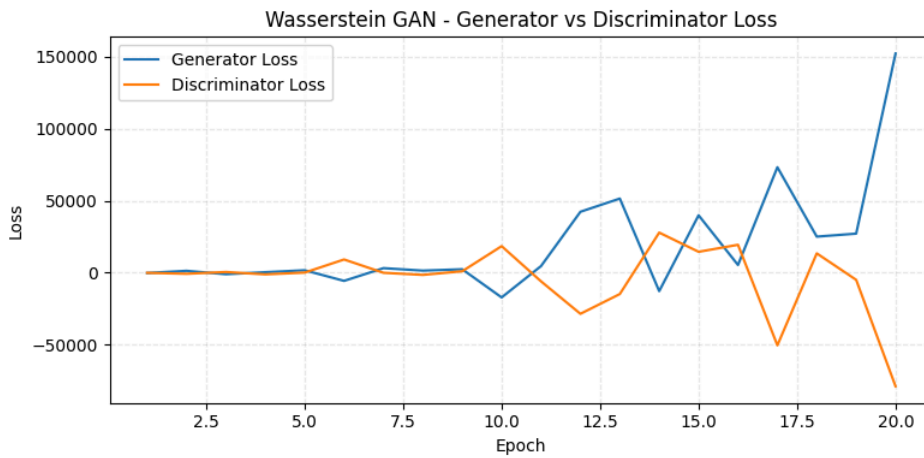
GAN variant: Wasserstein GAN: epoch 20/20

3500/3500 364s 104ms/step - loss_G: 152187.5625 - loss_D: -78895.6016

GAN variant: Wasserstein GAN epoch 20 complete - loss_G: 152187.562, loss_D: -78895.602

Loaded best Wasserstein GAN weights from epoch 1 with |G-D| diff 125.9121





Wasserstein GAN metrics:
 FID: 276.720, MiFID: 294.169,
 Memorization: 17.448

Least Squares (best at epoch 17): Losses oscillate but gradually narrow after epoch ~14; by epoch 17 both G and D hover near 0.9–1.0, suggesting the adversaries reach a modest balance after earlier instability (notably the sharp D spike near epoch 8).

Hinge (best at epoch 2): Early losses are already relatively aligned; after epoch 2 both G and D swing widely up to ~5, showing less stable updates and no clear convergence trend, so the early checkpoint captures the most stable point.

Wasserstein (best at epoch 1): Loss magnitudes explode and flip sign after a few epochs, with large swings past $\pm 100k$, indicating either gradient explosion or poor critic clipping/penalty; the earliest epoch is the safest checkpoint before divergence dominates.

Results & Analysis

Across the five Monet generators, the **Least Squares GAN** clearly leads with the lowest FID 176.8 / MiFID 192.2 and memorization 15.4, outperforming both the untuned (FID 228.6 / MiFID 245.1 / mem 16.5) and tuned baselines (FID 237.7 / MiFID 253.7 / mem 16.0). The tuned variant slightly reduces memorization relative to base but doesn't beat it on FID/MiFID, implying regularization helps diversity but not fidelity. Hinge and Wasserstein GANs underperform on both fidelity metrics (FID ~276–277, MiFID ~293–294) and exhibit higher or comparable memorization (16.5 and 17.4), suggesting their training was less stable or converged to poorer optima for this task. Measurements are from 128 generated vs. 128 real samples per model.

```
In [19]: # Aggregate metrics from every trained model
all_model_metrics = []
all_model_metrics.append(base_model_record)
all_model_metrics.append(tuned_model_record)
for result in gan_variant_results:
    all_model_metrics.append({
        'name': result['name'],
        'weights_path': result.get('weights_path'),
        'metrics': result['metrics'],
        'epoch': result['best_epoch'],
    })

best_model_record = None
for record in all_model_metrics:
    if not record or record.get('weights_path') is None or record.get('metrics') is None:
        continue
    if best_model_record is None or record['metrics']['fid'] < best_model_record['metrics']['fid']:
        best_model_record = record

best_model_record
```

```
Out[19]: {'name': 'Least Squares GAN',
'weights_path': './checkpoints/least_squares_gan_generator.weights.h5',
'metrics': {'fid': 176.8008276913249,
'mifid': 192.22809858060714,
'memorization': 15.427270889282227,
'num_real': 128,
'num_generated': 128},
'epoch': 17}
```

Least Squares GAN achieved its best checkpoint at epoch 17, posting FID 176.8, MiFID 192.2, and memorization 15.43 on 128 real vs 128 generated samples.

Create Kaggle Submission

- All of the photos files are converted to Monet's for submission to the Kaggle competition

```
In [ ]: # Lazily instantiate a generator from saved weights
def load_generator_from_weights(weights_path):
    generator = build_generator()
    generator.load_weights(weights_path)
    return generator
```

```

# Convert full dataset to Monet outputs and archive them
def convert_dataset_to_monet(photo_paths, generator, output_zip_path='images.zip'):
    temp_dir = tempfile.mkdtemp(prefix='monet_convert_')
    try:
        for path in photo_paths:
            photo = load_image(path)
            monet_pred = generator(tf.expand_dims(photo, axis=0), training=False)
            monet_img = tf.squeeze(monet_pred, axis=0)
            monet_img = denormalize_image(monet_img).numpy()
            file_name = os.path.basename(path)
            output_path = os.path.join(temp_dir, file_name)
            tf.keras.utils.save_img(output_path, monet_img)
        with zipfile.ZipFile(output_zip_path, 'w', zipfile.ZIP_DEFLATED) as zipf:
            for file_name in sorted(os.listdir(temp_dir)):
                file_path = os.path.join(temp_dir, file_name)
                zipf.write(file_path, arcname=file_name)
    finally:
        shutil.rmtree(temp_dir, ignore_errors=True)
    return output_zip_path

# Create submission images.zip file
output_zip_path = './images.zip'
conversion_generator = load_generator_from_weights(best_model_record['weights_path'])
zip_path = convert_dataset_to_monet(photo_images_paths, conversion_generator, output_zip_path)
metrics = best_model_record['metrics']
print(f"Converted {len(photo_images_paths):,} photos using {best_model_record['name']} (epoch {best_model_record['epoch']}) -> FID {metrics['fid']:.3f}")
del conversion_generator
gc.collect()

```

Converted 3500 photos using Least Squares GAN (epoch 17) -> FID 176.801, MiFID 192.228. Archive saved to ./images.zip

Out[]: 1338

```

In [3]: # Image of Kaggle submission
Image(url='submission_screenshot.png')

```

Out[3]:

I'm Something of a Painter Myself

Use GANs to create art - will you be the next Monet?



Overview Data Code Models Discussion Leaderboard Rules Team **Submissions**

Submissions

All Successful Errors

Recent ▾

Submission and Description

Public Score ⓘ



notebook3570aa03a8 - Version 6

Succeeded · 9m ago

83.16849

Conclusion

I built and compared five Monet generators (base, tuned, Least Squares GAN, Hinge GAN, Wasserstein GAN); the Least Squares model led with FID 176.8 / MiFID 192.2 / memorization 15.4, beating the base (FID 228.6) and tuned (FID 237.7) baselines, while Hinge and Wasserstein lagged near FID ~277 with higher memorization.

A key takeaway is that the loss formulation drove the biggest gains: the least-squares objective improved both fidelity and diversity, whereas tuning the baseline only modestly reduced memorization without matching its fidelity; stability-focused objectives outperformed simple hyperparameter tweaks.

The biggest miss was unstable training for Hinge and Wasserstein—loss curves swung or diverged, pointing to insufficient critic regularization/clipping or learning-rate control, which likely prevented convergence to higher-quality samples.

To improve, add stronger critic regularization (gradient penalty or spectral norm) for Hinge/WGAN, use adaptive learning-rate schedules with early stopping on FID, increase sample counts and evaluation robustness, and explore architectural upgrades (e.g., ResNet blocks, attention, normalization tweaks) while keeping least-squares as the stable reference.